



Practica 1

[Practica-1---Variables-compartidas.pdf](#)

1. Para el siguiente programa concurrente suponga que todas las variables están inicializadas en 0 antes de empezar. Indique cual/es de las siguientes opciones son verdaderas:

- a. En algún caso el valor de x al terminar el programa es 56.
- b. En algún caso el valor de x al terminar el programa es 22.
- c. En algún caso el valor de x al terminar el programa es 23.

P1:: If (x = 0) then y:= 4*2; x:= y + 2;	P2:: If (x > 0) then x:= x + 1;	P3:: x:= (x*3) + (x*2) + 1;
--	--	---------------------------------------

P1:

- 1. Load y, r1
- 2. Add 2, r1
- 3. Store r1, x

P2:

- 4. Load x, r2
- 5. Add 1, r2
- 6. Store r2, x

P3:

- 7. Load x, r3

8. Load x, r4
9. Multiplication r3, 3
10. Multiplication r4, 2
11. Add r3,r4,r5
12. Add 1, r5
13. Store r5, x

a) Verdadero, ejecutando P1, P2, P3 -> X = 56

P1:

1. Load y, r1 -> 8
2. Add 2, r1 -> 10
3. Store r1, -> x = 10

P2:

4. Load x, r2 -> r2 = 10
5. Add 1, r2 -> 11
6. Store r2, x -> r2 = 11

P3:

7. Load x, r3 -> r3 = 11
8. Load x, r4 -> r3 = 11
9. Multiplication r3, 3 -> r3 = 33
10. Multiplication r4, 2 -> r4= 22
11. Add r3,r4,r5 -> r5 = 33 + 22 = 55
12. Add 1, r5 -> 56
13. Store r5, x -> x = 56

b) Verdadero, si se sigue este orden de ejecucion

P3:

7. Load x, r3 -> r3 = 0

P1:

1. Load y, r1 -> r1 = 8
2. Add 2, r1 -> r1 = 10
3. Store r1, x -> x= 10

P3:

8. Load x, r4 -> r4 = 10
9. Multiplication r3, 3 -> r3 = 0
10. Multiplication r4, 2 -> r4= 20
11. Add r3,r4,r5 -> r5 = 0 + 20 = 20
12. Add 1, r5 -> 21

13. Store r5, x -> x = 21

P2:

4. Load x, r2 -> r2 = 21

5. Add 1, r2 -> r2 = 22

6. Store r2, x -> x = 22

c) Verdadero, si se sigue este orden de ejecucion

P3:

7. Load x, r3 -> r3 = 0

P1:

1. Load y, r1 -> r1 = 8

2. Add 2, r1 -> r1 = 10

3. Store r1, x -> x = 10

P2:

4. Load x, r2 -> r2 = 10

5. Add 1, r2 -> r2 = 11

6. Store r2, x -> x = 11

P3:

8. Load x, r4 -> r3 = 11

9. Multiplication r3, 3 -> r3 = 0

10. Multiplication r4, 2 -> r4 = 22

11. Add r3, r4, r5 -> r5 = 0 + 22 = 22

12. Add 1, r5 -> 23

13. Store r5, x -> x = 23

2. Realice una solución concurrente de grano grueso (utilizando <> y/o <await B; S>) para el siguiente problema. Dado un número N verifique cuántas veces aparece ese número en un arreglo de longitud M. Escriba las pre-condiciones que considere necesarias.

Precondiciones:

-M>0

-Arreglo inicializado con M elementos cargados

```
int total = 0; int arreglo[M]; int N;
```

```
Process Contador [id: 0..M-1] {  
    int suma = 0;
```

```

if (arreglo[id] == N) {
    suma = suma + 1;
}
<total = total + suma>;
}

```

3. Dada la siguiente solución de grano grueso:

- a. Indicar si el siguiente código funciona para resolver el problema de Productor/Consumidor con un buffer de tamaño N. En caso de no funcionar, debe hacer las modificaciones necesarias.

int cant = 0; int pri_ocupada = 0; int pri_vacia = 0; int buffer[N];	
Process Productor:: { while (true) { <i>produce elemento</i> <await (cant < N); cant++> buffer[pri_vacia] = <i>elemento</i> ; pri_vacia = (pri_vacia + 1) mod N; } }	Process Consumidor:: { while (true) { <await (cant > 0); cant-- > <i>elemento</i> = buffer[pri_ocupada]; pri_ocupada = (pri_ocupada + 1) mod N; <i>consume elemento</i> } }

-Problemas:

El buffer podría estar vacío y Cant podría ser 0 al inicio, por lo tanto podría ser menor a N, lo que generaría que el Productor aumente cant 1 y eso podría generar que el procesador le de el control a Consumidor (sin que llegue a poner nada en el buffer) y cuando consumidor chequee $\text{cant} > 0$ le va a dar true, va a decrementar cant y el Productor jamás había llegado a poner el elemento, por lo que cuando lea el elemento del buffer no va a tener nada

-Solucion:

Se tiene que asegurar el incremento/decremento de cant antes y post llenado/vaciado del buffer, es decir, que se haya PUESTO en el buffer el elemento si se aumento Cant y luego liberar la sincronización y viceversa, que se haya SACADO del buffer el elemento si se decremento Cant y luego liberar la sincronización. Con esto nos evitaríamos que Consumidor lea vacío del buffer porque Productor no puso nada y aumento en cant, o el caso contrario, que Productor intente llenar un buffer que todavía no fue vaciado

```

int cant= 0; int pri_ocupada= 0; int pri_vacia=0; int
buffer[N];

Process Productor::{
    //produce elemento
    <await (cant < N); cant++;
    buffer[pri_vacia] = elemento;>
    pri_vacia = (pri_vacia + 1) MOD N;
}

Process Consumidor::{
    while(true) {
        <await(cant > 0); cant -;
        elemento= buffer[pri_ocupada];>
        pri_ocupada= (pri_ocupada) + 1) mod N;
        //consume elemento
    }
}

```

b. Modificar el código para que funcione para C consumidores y P productores.

-Se tiene que asegurar que las variables compartidas no sean interferidas al mismo tiempo, por lo que el await se alarga hasta luego de modificar pri_vacia y pri_ocupada. Mas de un Productor puede tocar la variable pri_vacia y poner a la vez un elemento en esa posicion y generaria superposicion de datos/elementos en el array y tambien mas de un Consumidor podria sacar del array de la misma pri_ocupada al mismo tiempo retirando 2 veces el mismo elemento que haya metido en el array

```

int cant= 0; int pri_ocupada= 0; int pri_vacia=0; int
buffer[N];

Process Productor::[id: 0..P-1]{
    //produce elemento
    <await (cant < N); cant++;

```

```

    buffer[pri_vacia] = elemento;
    pri_vacia = (pri_vacia + 1) MOD n;>
}
}

Process Consumidor::[id: 0..C-1]{
    while(true) {
        <await(cant > 0); cant -;
        elemento= buffer[pri_ocupada;>
        pri_ocupada= (pri_ocupada) + 1) mod N;>
        //consume elemento
    }
}

```

4. Resolver con SENTENCIAS AWAIT (<> y <await B; S>). Un sistema operativo mantiene 5 instancias de un recurso almacenadas en una cola, cuando un proceso necesita usar una instancia del recurso la saca de la cola, la usa y cuando termina de usarla la vuelve a depositar.

```

/* Solucion 1: */
cola C;
int N = 5;

Process OS[id:0..N-1]{
    while(true){
        <await (N > 0);
        recurso = Sacar(C);
        N = N-1;>
        <Agregar(C, recurso)
        N = N + 1;>
    }
}

/* Solucion 2: */
cola C;
int N = 5;

```

```

Process OS[id: 0..N-1] {
    while(true){
        <await (not C.empty())
        recurso = Sacar(C);>
        <Agregar (C, recurso)>
    }
}

```

5. En cada ítem debe realizar una solución concurrente de grano grueso (utilizando `<>` y/o `<await B; S>`) para el siguiente problema, teniendo en cuenta las condiciones indicadas en el ítem. Existen N personas que deben imprimir un trabajo cada una.

- a. Implemente una solución suponiendo que existe una única impresora compartida por todas las personas, y las mismas la deben usar de a una persona a la vez, sin importar el orden. Existe una función `Imprimir(documento)` llamada por la persona que simula el uso de la impresora. Sólo se deben usar los procesos que representan a las Personas.

```

Process PersonaA[id: 0..N-1]{
    <Imprimir(documento)>
}

```

- b. Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.

```

cola C;
int siguiente = -1;

Process PersonaB[id: 00..N-1]{
    // si siguiente = -1 voy yo, actualizo id, sino m
    e agrego a la fila
    <if (siguiente == -1) siguiente = id
    else Agregar(C, id)>
    <await siguiente == id>;
    // uso impresora
}

```

```

    documento = Imprimir(documento);
    // reviso si no hay nadie en la fila, sino actual
    izo siguiente
    <if (empty(C)) siguiente = -1
        else Siguiente = Sacar(C)>
}

```

- c. **Modifique la solución de (a) para el caso en que se deba respetar el orden dado por el identificador del proceso (cuando está libre la impresora, de los procesos que han solicitado su uso la debe usar el que tenga menor identificador).**

```

/*Entiendo que es la misma cola de prioridad que la e
xplicacion Practica */
colaEspecial C;
int siguiente = -1;

Process PersonaB[id: 00..N-1]{
    <if (siguiente = -1) siguiente = id
        // Aca cuando lo pone lo pone segun orden de
        id asumiendo lo de arriba
        else Agregar(C, id)>
    <await siguiente == id>;
    documento = Imprimir(documento);
    <if (empty(C)) siguiente = -1
        // Aca cuando saca deberia sacar al de menor
        ID asumiendo lo de arriba
        else Siguiente = Sacar(C)>
}

```

- d. **Modifique la solución de (b) para el caso en que además hay un proceso Coordinador que le indica a cada persona que es su turno de usar la impresora.**

```

cola C;
int siguiente = -1;
bool turno = false;

```

```

Process PersonaD[id: 00..N-1]{
    <Agregar(C, id)>
    <await (siguiente == id>;
    documento = Imprimir(documento)
    turno = false;
}

Process Coordinador{
    while(true){
        <await (not turno && not C.empty());
        siguiente = Sacar(C);>
        turno = true;
    }
}

```

6. Dada la siguiente solución para el Problema de la Sección Crítica entre dos procesos (suponiendo que tanto SC como SNC son segmentos de código finitos, es decir que terminan en algún momento), indicar si cumple con las 4 condiciones requeridas:

<pre> int turno = 1; Process SC1:: { while (true) { while (turno == 2) skip; SC; turno = 2; SNC; } } </pre>	<pre> Process SC2:: { while (true) { while (turno == 1) skip; SC; turno = 1; SNC; } } </pre>
---	--

-Las 4 condiciones serian exclusion mutua, ausencia de deadlock, ausencia de demora innecesaria y eventual entrada

-Exclusion mutua → Se cumple ya que solo uno de los 2 procesos va a estar tocando la variable turno, nunca van a estar los 2 a la vez

-Ausencia de deadlock → Se cumple ya que no hay ninguna alternativa en la que SC1 se queda esperando que SC2 haga algo que lo destrabe ni SC2 se queda esperando que SC1 haga algo que lo destrabe, nunca se quedan los dos bloqueados

-Ausencia de Demora Innecesaria → En teoría lo cumple ya que cada proceso simplemente espera que el otro modifique la variable compartida y luego le llega su turno. Si se tendría en cuenta el hecho de que SC2 quiera acceder a la sección crítica y no puede debido a que SC1 todavía modifica turno entonces si la hay(?)

-Eventual Entrada → Se cumple, el turno altera la entrada de los dos procesos por lo que garantiza que ambos tendrán la oportunidad de ingresar a la sección crítica

7. **Desarrolle una solución de grano fino usando sólo variables compartidas (no se puede usar las sentencias *await* ni funciones especiales como *TS* o *FA*). En base a lo visto en la clase 3 de teoría, resuelva el problema de acceso a sección crítica usando un proceso coordinador. En este caso, cuando un proceso SC[i] quiere entrar a su sección crítica le avisa al coordinador, y espera a que éste le dé permiso. Al terminar de ejecutar su sección crítica, el proceso SC[i] le avisa al coordinador. Nota: puede basarse en la solución para implementar barreras con "Flags y coordinador" vista en la teoría 2.**